

Аннотация

Отчет 45 с., 20 рис., 35 источников. Ключевые слова: верификация, сети Петри, вложенные сети Петри, параллелизм, проверка моделей, развертки, мультиагентные системы.

Темой данного исследования является исследования метода разверток в применении к верификации вложенных сетей Петри.

Формализм вложенных сетей Петри используется для моделирования мультиагентных систем – систем, состоящих из набора автономных агентов, имеющих свое собственное поведение, взаимодействующие через механизмы синхронизации. Засчет наличия автономного поведения, мультиагентные системы имеют являются сильно распараллеленными.

При верификации параллельных и распределенных система часто возникает проблема экспоненциального роста пространства достижимых состояний системы. Одной из причин этого является наличие большого числа промежуточных состояний, получаемых в результате рассмотрения всех возможных последовательных исполнений системы – интерливинга (interleaving). Один из способов решения этой проблемы использует построение (конечных префиксов) разверток сетей.

Развертки сетей (net unfoldings) используются для описания поведения системы в семантике истинного параллелизма. В случае высокопараллельных систем развертки (а точнее их конечные префиксы) имеют размер меньший чем графы достижимости, тем самым позволяя частично обойти проблему взрывоподобного роста числа достижимых состояний.

Основным результатом данной работы является определение разверток и ветвящихся процессов для консервативных вложенных сетей Петри, моделирующих системы с неизменным множеством агентов. Это определение используется для построения основанного на развертках алгоритма верификации важных поведенческих свойств для консервативных вложенных сетей Петри.

Результаты данных исследований представлены в докладе, принятом на семинар Verification and Program Transformation, в рамках международной конференции Computer Aided Verification 2014, которая будет проходить в Вене в июле этого года.

Оглавление

Введение	4
1 Сети Петри и развертки сетей	6
1.1 Сети Петри	6
1.2 Развертки сетей	9
1.3 Библиографический комментарий	13
2 Вложенные сети Петри и модели мультиагентных систем	15
2.1 Определение вложенных сетей Петри	16
2.2 Примеры моделирования	19
2.3 Библиографический комментарий	22
3 Развертки вложенных сетей Петри	23
3.1 Покомпонентная развертка сетей	23
3.2 Ветвящиеся процессы вложенных сетей Петри	25
4 Верификация с помощью разверток	34
4.1 Канонические префиксы разверток	34
4.2 Алгоритм генерации канонического префикса	37
Заключение	41
Список использованных источников	42

Введение

Данная работа посвящена тематике верификации формальных моделей распределенных мультиагентных систем.

Мультиагентные системы – это системы, которые состоят из нескольких отдельных автономных сущностей – агентов, имеющих свои цели и задачи, и коммуницирующих друг с другом.

В современном мире, распределенные мульти-агентные и параллельные системы становятся все более популярными. Очень часто мы не замечаем их присутствие, но современные настольные и портативные компьютеры работают на мультиядерных процессорах, и даже наша бытовая техника часто контролируется параллельными системами. Цена ошибки в таких системах может быть очень велика. Вследствие этого возрастает спрос на методы верификации таких систем. В большинстве случаев, невозможно верифицировать реальную систему, поэтому объектами верификации являются модели систем. Таким образом рабочий процесс разработки распределенных параллельных систем выглядит следующим образом: моделирование, верификация модели, имплементация. Это позволяет обнаруживать ошибки на ранних стадиях разработки.

Популярным формализмом для моделирования распределенных систем являются сети Петри. Вложенные сети Петри (nested Petri nets, NP-сети) [22, 35] являются расширением формализма сетей Петри высокого уровня, в соответствии с парадигмой “сети внутри сетей”. Подход “сети внутри сетей” широко изучается в литературе, как подход для моделирования активных объектов, мобильности и динамичности в распределенных системах [1, 15, 20, 34].

NP-сети – это удобный формализм для моделирования динамических систем с взаимодействующими агентами: каждый агент представляется отдельной элементной фишкой, в то время как агенты расположены в системной сети. Смежные уровни в NP-сетях взаимодействуют друг с другом с помощью механизма вертикальной синхронизации (одновременное срабатывание переходов в системной сети и в элементных сетях). Из-за гибкой связанности многоуровневой структуры NP-сети являются эффективным формализмом для моделирования мультиагентных систем ([7, 13, 24, 23, 3] для примера).

Мультиагентные системы являются сильно-параллельными (т.е. содержат высокий процент параллелизма) за счет того, что агенты имеют почти независимое поведение. При верификации систем параллельные поведения часто симулируют с помощью недетермини-

рованного последовательного исполнения (как если бы параллельная программа исполнялась на компьютере с одним процессором). Такой подход заметно упрощает семантику системы, но порождает другую проблему – огромное количество interleavings – способов последовательного исполнения событий. Это приводит к так называемой проблеме взрывообразного роста числа достижимых состояний системы (state space explosion problem), когда пространство состояний системы возрастает экспоненциально при добавлении новых компонент. Для решения этой проблемы были разработаны различные методы, основывающиеся на семантике т.н. “истинного параллелизма” (true concurrency semantics). Один из самых популярных методов основан на конечных префиксах разверток сетей (finite prefixes of unfoldings). Все последовательные исполнения системы можно собрать в дерево вычислений (computation tree) – дерево, в котором вершины являются глобальными состояниями системы, а дуги – событиями, которые связывают состояния со следующими возможными состояниями. Развертка сети [29] является в свою очередь собранием параллельных исполнений (nonsequential process) системы. В случаях когда параллельная система не имеет бесконечно длинных исполнений, размер развертки может быть сильно меньше размера графа достижимости, за счет того, что в развертке не учитываются промежуточные состояния, которые возникают за счет interleaving’a. В случае когда параллельная система имеет бесконечные исполнения, часто развертку можно сократить до какого-то конечного префикса.

В данной работе представлено определение развертки и алгоритм построения конечных префиксов развертки для подкласса вложенных сетей Петри.

В первой главе представлена общие понятия касающиеся сетей Петри и разверток. Во второй главе описан формализм вложенных сетей Петри и подкласс консервативных NP-сетей. В третьей главе содержится основной вклад автора, а именно, определение ветвящихся процессов и разверток для консервативных вложенных сетей Петри. В последней главе представлен алгоритм построения конечных префиксов разверток консервативных NP-сетей, который основан на теории канонических префиксов и контекста срезания (cutting context). В работе показано, что определение разверток для консервативных NP-сетей удовлетворяет фундаментальному свойству разверток. Данное определение позволяет переносить результаты теории канонических префиксов и контекста срезания на консервативные вложенные сети Петри.

1 Сети Петри и развертки сетей

В этой главе будут рассмотрены базовые понятия, касающиеся тематики работы и используемые в тексте: сети Петри и развертки сетей.

1.1 Сети Петри

Сети Петри – это формализм, используемый для описания распределенных и параллельных систем. Сеть Петри является направленным взвешенным графом, имеющим вершины двух типов: позиции (графически обозначаются кружками) и переходы (графически обозначаются прямоугольниками). Внутри позиций могут находиться черные фишки. Часто, фишки символизируют ресурсы или порядок исполнения (control flow), а переходы – возможные события. Переходы в сети Петри могут срабатывать, изменяя распределение фишек по позициям сети. Распределение фишек по позициям сети называется *маркировкой* сети и символизирует состояние системы, которая моделируется с помощью сети Петри.

Формальное определение сетей Петри и их поведения требует введения понятия мультимножества.

Мультимножества. Пусть S – конечное множество. *Мультимножество* m над множеством S является функцией $m : S \rightarrow \mathbb{N}$. Неформально говоря, мультимножество является множеством, в котором некоторые элементы могут присутствовать более одного раза. Для двух мультимножеств m, m' пишут $m \subseteq m'$ если $\forall s \in S : m(s) \leq m'(s)$. Функции суммы и вычитания мультимножеств определяются стандартным образом: $\forall s \in S : (m + m')(s) = m(s) + m'(s)$, $(m - m')(s) = \max(0, m(s) - m'(s))$.

Если $f : S \rightarrow S'$ – функция, то образом мультимножества m относительно f называется мультимножество $f(m)$, определенное следующим образом: $f(m)(s') = \sum_{s \in f^{-1}(s')} m(s)$.

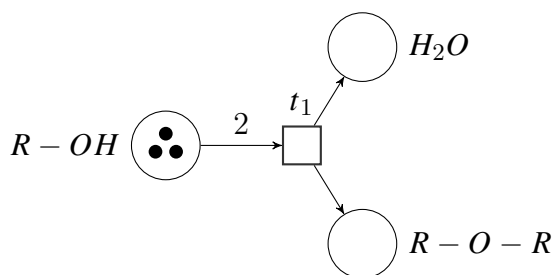


Рис. 1: Химический процесс представленный в виде сети Петри

Определение 1.1 (Сеть Петри).

(Помеченная) Сеть Петри – это четверка (P, T, F, λ) , где

- P, T – раздельные множества позиций и переходов, соответственно;
- $F \subset (P \times T) \cup (T \times P) \rightarrow \mathbb{N}$ – отношения потока;
- $\lambda : T \rightarrow \Lambda$ – частичная функция пометки.

Если сеть Петри не помечена, то последний компонент λ опускают.

Маркировкой сети $N = (P, T, F, \lambda)$ является мультимножество над множеством позиций P . Маркированная сеть Петри (N, M_0) – это сеть, вместе с ее начальной маркировкой.

Для перехода $t \in T$, дуга (x, t) называется *входной дугой*, и дуга (t, x) – *выходной*. Для каждой вершины $x \in P \cup T$ определено *пред-множество* $\bullet x = \{y \mid (y, x) \in F\}$ и *пост-множество* $x^\bullet = \{y \mid (x, y) \in F\}$.

Говорят, что переход t в сети Петри $N = (P, T, F)$ *активен* в маркировке M , если $\bullet t \subseteq M$. Активный переход может сработать, в результате чего получается новая маркировка $M' = M - \bullet t + t^\bullet$ (обозначается как $M \xrightarrow{t} M'$). Маркировка M называется *достижимой*, если существует (возможно пустая) последовательность срабатываний $M_0 \xrightarrow{t_1} M_1 \xrightarrow{t_2} M_2 \rightarrow \dots \rightarrow M$ из начальной маркировке сети в M . Через $\mathcal{RM}(N)$ обозначают множество всех достижимых маркировок N .

Маркировка M называется *безопасной*, если для всех позиций $p \in P$, $M(p) \leq 1$. Сеть Петри называется *безопасной*, если все ее достижимые маркировки $M \in \mathcal{RM}(N)$ безопасны. В этой работе рассматриваются только безопасные сети Петри.

Граф достижимости маркированной сети Петри – это помеченный направленный граф, вершинами которого являются достижимые маркировки сети, а дугами – переходы. При этом, между двумя маркировками M, M' существует дуга t тогда и только тогда, когда $M \xrightarrow{t} M'$.

Анализ поведенческих свойств (верификация) сетей Петри. Одним из преимуществ использования математических формализмов для моделирования систем является то, что модели систем поддаются формальной верификации. Для верификации моделей, представленных в виде сетей Петри, используются как структурные (анализ инвариантов, ловушки), так и т.н. методы проверки моделей, которые основываются на анализе графа достижимости. Можно привести некоторые примеры свойств, которые поддаются проверке такими методами:

- Свойство достижимости: какая-то маркировка m (какое-то состояние системы) является достижимой в сети.

- Отсутствие дедлоков в системе: в любой достижимой маркировке в сети может сработать хотя бы один переход.
- Свойство срабатывания: какое-то событие, обозначенное переходом t , может произойти в системе, т.е. существует такая достижимая маркировка, при которой переход t является активным.

Однако, анализ полного пространства состояний часто бывает невозможен, ввиду большого размера графа достижимости. При композиции параллельных компонент количество допустимых состояний системы может возрасть экспоненциально.

Такой рост объясняется тем, что в графе достижимости учитываются все возможные последовательные исполнения параллельных компонент – interleavings. (Можно провести аналогию с анализом пространства состояний и анализом поведения мультипоточной программы на одном процессоре.)

Для примера можно рассмотреть сеть Петри показанную на рис. 2. Граф достижимости такой сети имеет 2^k вершин, где k количество параллельных компонент в системе. Все последовательные исполнения этой сети равноправны и одинаковы в том смысле, что срабатывание перехода в одной из компонент не зависит от срабатывания переходов в других компонентах. Таким образом, порядок срабатывания переходов в N_X не играет никакой роли.

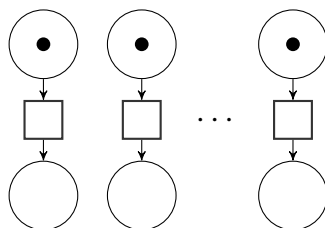


Рис. 2: Сеть Петри N_X . Размер граф достижимости такой сети возрастает экспоненциально с добавлением параллельных компонент.

Некоторые методы решения проблемы экспоненциального роста размера пространства состояний базируются на семантике истинного параллелизма. Семантика истинного параллелизма – эта семантика, которая различает параллельное срабатывание двух событий a и b ($a \parallel b$, на языке процессных алгебр) от недетерминированного выбора срабатывания событий a и b или срабатывания b и a ($a.b + b.a$).

Один из таких подходов – развертки (unfoldings) – будет рассмотрен в следующей части.

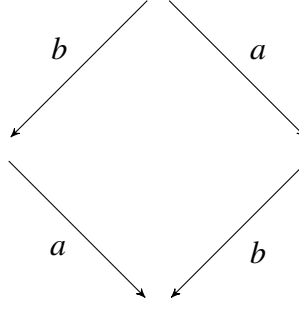


Рис. 3: $a.b + b.a$

1.2 Развертки сетей

Развертки используются для задания параллельной семантики сетей Петри, и они соответствуют полному набору параллельных исполнений сети. Формально, развертки задаются через специальный подкласс сетей Петри, которые носит название сети событий (occurrence nets). В этой главе мы рассмотрим определения разверток и основные факты о них.

Пусть $N = (P, T, F)$ – сеть Петри. На вершинах $P \cup T$ сети определены следующие отношения.

1. Отношение *зависимости*, обозначаемое $<$, – это транзитивное замыкание F , а $\leq$ – рефлексивное замыкание $<$.
2. Отношение *конфликта*. Две вершины $x, y \in P \cup T$ находятся в конфликте, если $x \# y \iff \exists t, t' \in T. t \neq t' \wedge \bullet t \cap \bullet t' \neq \emptyset \wedge t \leq x \wedge t' \leq y$.
3. Отношение *параллельности*, обозначаемое как co : две вершины параллельны, если они не в конфликте, и ни одна из них не зависит от другой.

Для множества вершин B пишут $co(B)$ если все вершины в B попарно параллельны.

Примеры двух вершин, зависящих одна от другой, конфликтующие друг с другом или параллельные друг другу можно найти на рис. 4 (слева), на рис. 4 (справа) или на рис. 5, соответственно.

Определение 1.2.

Сеть событий (occurrence net) – это безопасная сеть Петри $ON = (B, E, G)$, где

1. В ON нет циклов.
2. $\forall p \in B: |\bullet p| \leq 1$.

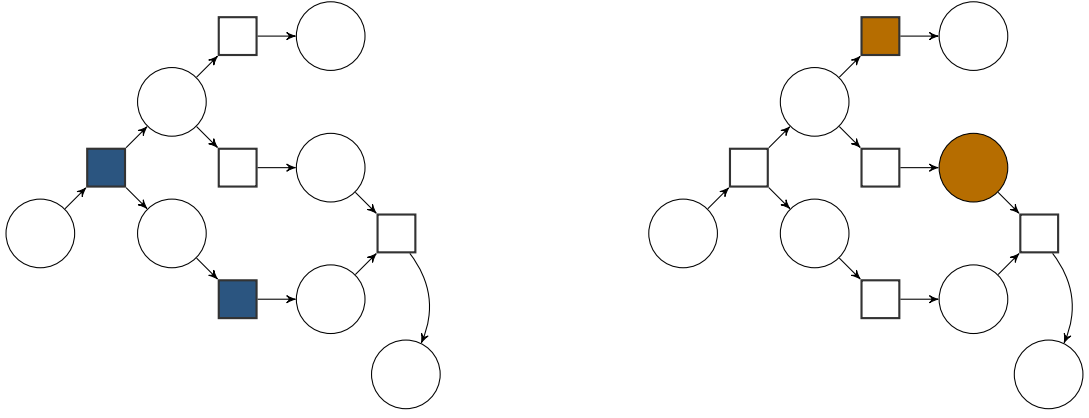


Рис. 4: Иллюстрация к отношениям зависимости и конфликта

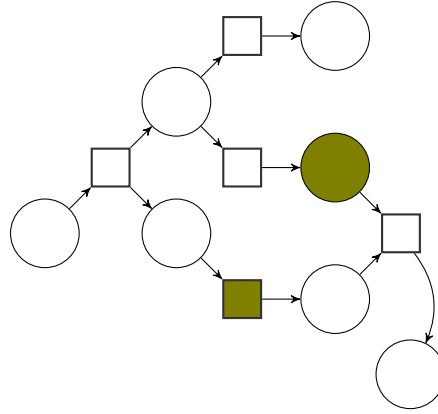


Рис. 5: Иллюстрация к отношению параллельности

3. $\forall x \in B \cup E$ множество $\{y \mid y < x\}$ конечно.

4. $\forall x \in B \cup E: \neg(x \# x)$.

Обычно позиции в сетях событий называют *условиями*, а переходы – *событиями*. Множество $B \cup E$ называют множеством *вершин*, и пишут $x \in ON$, если $x \in B \cup E$.

Также нам пригодится следующая теорема, характеризующая отношение параллельности:

Теорема 1.1.

Пусть S – множество позиций в маркированной сети событий ON . Следующие утверждения равносильны:

- Существует достижимая маркировка M , в которой $S \subseteq M$.
- $co(S)$

То есть, множество условий S может быть покрыто какой-либо маркировкой тогда и только тогда, когда S попарно параллельно.

В сетях событий также определяют конфигурации – бесконфликтные множества вершин, замкнутые относительно $<$:

Определение 1.3.

Пусть ON – сеть событий. Конфигурацией ON называется множество вершин C , для которого выполняется следующее:

1. $\forall x, y \in C, \quad \neg(x \# y)$;
2. $\forall x \in C$, если $y < x$, то $y \in C$.

Для каждой вершины e конфигурация $[e] = \{x \mid x \leq e\}$ называется *локальной конфигурацией*. Также, если C – конфигурация, а E' – множество событий, то под $C \oplus E'$ понимается множество $C \cup E' \cup E'^\bullet$, если оно является конфигурацией, и если $C \cap E' = \emptyset$. В таком случае C называют *префиксом*, а E' – *суффиксом*.

Прежде чем перейти к определению ветвящихся процессов, надо определить каноническое представление сетей событий. По-существу, определяя каноническое представление сетей событий и устанавливая одну четкую схему образования имен для позиций и переходов сети, мы “склеиваем” изоморфные сети в одну, избавляясь тем самым от проблем связанных с рассмотрением изоморфных, но разных сетей.

Множество C *канонических имен* для маркированной сети Петри $N = (P, T, F, M_0)$ является наименьшим множеством имен таким, что если $x \in P \cup T$, а A – конечное подмножество C , то $(A, x) \in C$.

Тогда C -сеть – это сеть событий (B, E, G) , где

- $B \cup E \subseteq C$.
- $\forall (A, x) \in B \cup E, \bullet(A, x) = A$.

Начальная маркировка C -сети является множеством $\{(\emptyset, x) \mid (\emptyset, x) \in B\}$.

Для каждой C -сети определяется функция (морфизм) h отображающая вершины C -сети на вершины сети N :

$$h((A, x)) = x \quad (1.1)$$

Если для вершины x выполняется $h(x) = y$, то мы говорим что вершина x помечена y , и обозначаем ее соответствующим образом графически.

Пусть S (конечное или бесконечное) множество C -сетей. Тогда объединение S определяется покомпонентно: $\bigcup S = (\bigcup_{(P,T,F,M) \in S} P, \bigcup_{(P,T,F,M) \in S} T, \bigcup_{(P,T,F,M) \in S} F, \bigcup_{(P,T,F,M) \in S} M)$.

Определение 1.4 (Ветвящийся процесс).

Для маркированной сети Петри $N = (P, T, F, M_0)$, множество ветвящихся процессов определяется индуктивно.

1. Сеть событий $(I, \emptyset, \emptyset)$, где $I = \{(\emptyset, p) \mid p \in M_0\}$, является ветвящимся процессом.
2. Пусть $\mathcal{B}_1$ – ветвящийся процесс, а M – достижимая маркировка $\mathcal{B}_1$, и $M' \subseteq M$, причем $h(M') = \bullet t$ для какого-либо t из T . Пусть $\mathcal{B}_2$ – сеть, полученная добавлением события (M', t) и условий $\{((M', t), p) \mid p \in t^\bullet\}$ к $\mathcal{B}_1$. Тогда $\mathcal{B}_2$ – ветвящийся процесс.
3. Пусть $\mathcal{B}\mathcal{B}$ (конечное или бесконечное) множество ветвящихся процессов. Тогда $\bigcup \mathcal{B}\mathcal{B}$ – ветвящийся процесс.

Примеры ветвящихся процессов для сети N_s (рис. 6) можно увидеть на рис. 7–9.

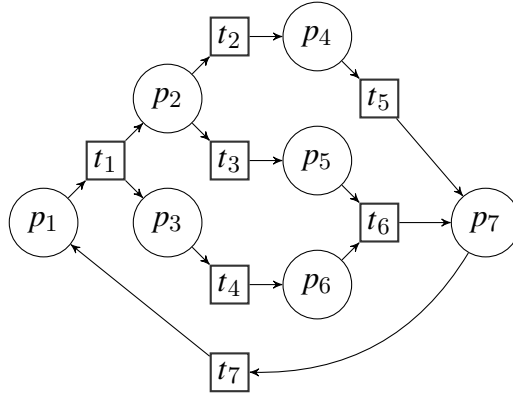


Рис. 6: Сеть Петри N_s

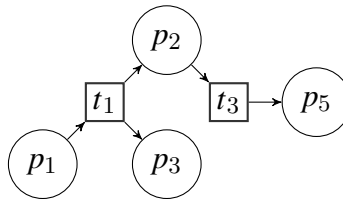


Рис. 7: Ветвящийся процесс BP_1 для сети N_s

Ветвящийся процесс $\mathcal{B}_1 = ((P_1, E_1, F_1), h_1)$ называется *префиксом* ветвящегося процесса $\mathcal{B}_2 = ((P_2, E_2, F_2), h_2)$ (обозначается как $\mathcal{B}_1 \sqsubseteq \mathcal{B}_2$), если $P_1 \subseteq P_2$ и $E_1 \subseteq E_2$.

Максимальный, с точки зрения $\sqsubseteq$, ветвящийся процесс называется *разверткой*.

Развертка полностью представляет поведение оригинальной сети, что выражается в следующей теореме.

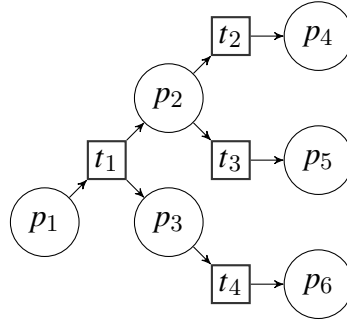


Рис. 8: Ветвящийся процесс BP_2 для сети N_s

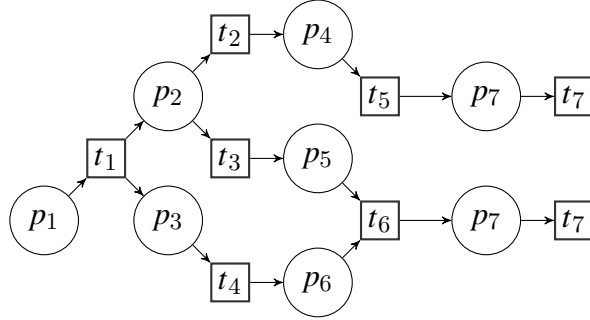


Рис. 9: Ветвящийся процесс BP_3 для сети N_s

Свойство 1.1 (Фундаментальное свойство разверток).

Пусть M – достижимая маркировка сети N . Пусть M_U – достижимая маркировка развертки $U(N)$, причем $h(M_U) = M$. Тогда

Если $M_U \xrightarrow{t_U} M'_U$ в $U(N)$, то существует срабатывание $M \xrightarrow{t} M'$ в N , такое что $h(t_U) = t \wedge h(M'_U) = M'$

Если $M \xrightarrow{t} M'$ в N , то существует срабатывание $M_U \xrightarrow{t_U} M'_U$ в $U(N)$, такое что $h(t_U) = t \wedge h(M'_U) = M'$

В следующей главе будет рассмотрен формализм вложенных сетей Петри. Именно для этого формализма в работе будет разработано понятие разверток. Для разверток вложенных сетей потребуются теорема схожая с 1.1, т.е. развертки вложенных сетей должны будут также полностью представлять поведение сети.

1.3 Библиографический комментарий

Сети Петри впервые были введены Карлом Адамом Петри и формализованы им в своей диссертации [30]. В настоящее время сети Петри являются популярным формализмом и объектом исследований. Заинтересованный читатель может обратиться к обзорным книгам по тематике [31, 6].

Изначально, развертки были введены Нильсеном, Плоткиным и Винскемом в [29]. Строго говоря, авторы не использовали современное определение развертки, а работали со схожим понятием “event structure”. Работа была посвящена семантике истинного параллелизма, и задачей авторов было установление соответствия между теорией сетей и теорией доменов Скотта. В статье не обсуждались вопросы верификации.

МакМиллан [27] был первым, кто применил теорию разверток для решения проблем верификации. В оригинальной работе МакМиллана обсуждалась верификация представленных в виде сетей Петри асинхронных электронных схем (asynchronous circuits), а именно проблема проверки срабатывания перехода. Алгоритм МакМиллана в последствии был улучшен и расширен для решения других проблем верификации [11, 28, 14, 5]. Алгоритм верификации, изначально предназначенный для безопасных сетей Петри, также был применен к другим моделям вычислений, таким как процессные алгебры [21, 2], синхронное произведение конечных автоматов [12], сети Петри со временем [4], и других.

Другой важной ветвью в развитии теории разверток, было формулирование понятия ветвящегося процесса (branching process) [8]. Если развертка, как объект, представляет из себя все возможные полные параллельные исполнения сети, то ветвящийся процесс – только частичные. Довольно часто, для решения проблем верификации требуется не вся развертка сети (которая может быть бесконечной), а только конечный ветвящийся процесс – префикс развертки.

Изначально, ветвящиеся процессы и алгоритмы нахождения конечных префиксов применялись к (безопасным) классическим сетям Петри, однако в последствии эти понятия были расширены и применены к сетям Петри высокого уровня [18]. В этой работе рассматривается расширения понятия ветвящегося процесса на вложенные сети Петри.

Общий метод сокращения разверток, который абстрагируется от конкретной информации, которую требуется сохранить в префиксе, был предложен в [19, 17]. Этот метод базируется на понятии *контекст срезания* (cutting context), о котором пойдет речь в главе 4.

Дополнительный обзор литературы по теории разверток сетей Петри можно найти в [9]. Книга [10] также является хорошим введением в тематику разверток.

2 Вложенные сети Петри и модели мультиагентных систем

Сети Петри являются удобным формализмом для описания параллельных систем. Тем не менее, за счет плоской структуры сетей, классические сети Петри не позволяют описывать простым образом модели мультиагентных систем: при моделировании сложных мультиагентных систем классическими сетями Петри (или даже иерархическими сетями Петри), модель становится большой и запутанной. Рудигер Вальк предложил концепцию т.н. “сетей внутри сетей” (“nets within nets”) [33]. В рамках этого подхода фишки в сети Петри имеют свою собственную структуру (которая также базируется на моделях сетей Петри) и свое поведение. Такая концепция позволяет с большим удобством моделировать системы с иерархией, мобильностью и инкапсуляцией.

На базе этого подхода существует несколько моделей, расширяющих классические сети Петри, и они различаются семантикой и свойствами. В этой работе рассматривается формализм вложенных сетей Петри. Вложенная сеть Петри состоит из системной сети (system net), в позициях которой находятся сетевые фишки (net tokens) – размеченные элементные сети (element nets). Сетевые фишки могут совершать действия независимо от системной сети, а могут координировать действия путем механизма *вертикальной синхронизации*.

Более конкретно, в данной работе рассматривается подкласс вложенных сетей Петри – консервативные вложенные сети Петри, основным свойством которого является то, что сетевые фишки не могут появляться или исчезать. Такое ограничение было выбрано автором по следующей причине. Классические развертки были определены для безопасных сетей Петри, и консервативные вложенные сети являются в свою очередь аналогом “безопасности” в мире вложенных сетей. Поскольку целью работы является предложить определение разверток для вложенных сетей, то именно этот класс был изначально выбран для рассмотрения.

Пример вложенной сети Петри можно увидеть на рис. 10. Эта сеть моделирует два процесса, пытающиеся получить доступ к одному ресурсу. Справа на рисунке показана системная сеть, в позиции *Res* которой находится ресурс – элементная сеть (обозначается не залитым кружком). Сама элементная сеть представлена на правой части рисунка. Как можно заметить, некоторые переходы в сетях помечены буквами. Таким образом обозначаются метки вертикальной синхронизации. Так, переход *Lock*₁ в системной сети может

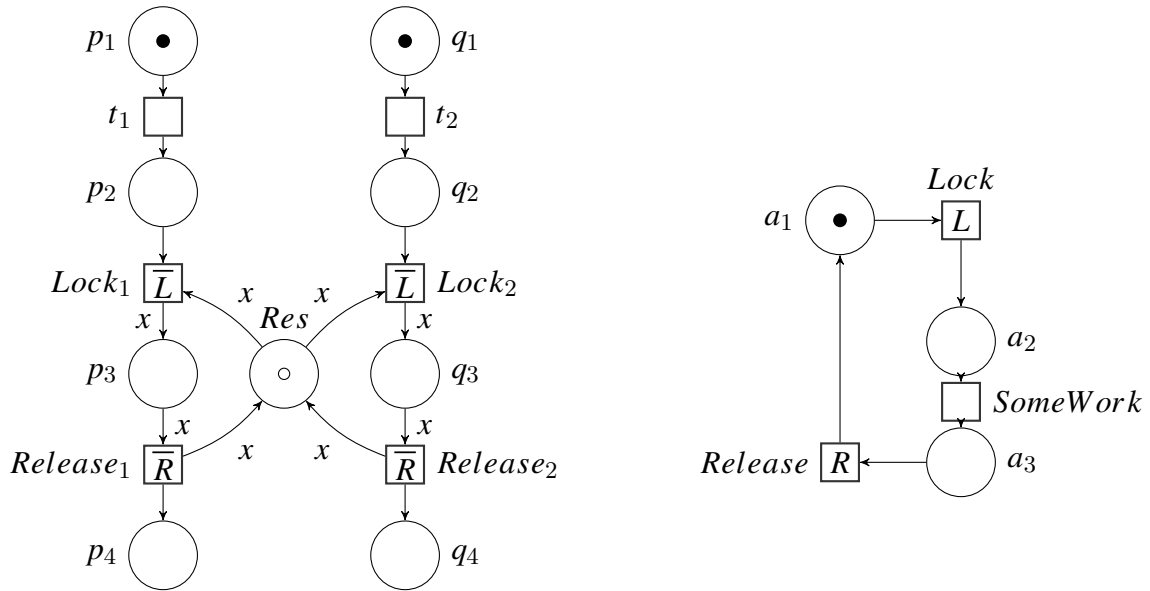


Рис. 10: NP-сеть NP_1

сработать только если в позиции p_2 находится черная фишка, а в позиции Res находится системная фишка, в которой активен переход $Lock$.

В следующей части будет формально описаны вложенные сети Петри и их поведение.

2.1 Определение вложенных сетей Петри

Для начала, зафиксируем некоторые множества. Пусть $Type$ – множество типов, Var – множество типизированных переменных, а Lab – множество пометок. Тогда определение вложенной сети Петри выглядит следующим образом.

Определение 2.1 (Вложенные сети Петри (NP-сети)).

Вложенная сеть Петри NP – это кортеж $(SN, (EN_1, \dots, EN_k), \nu, \lambda, W)$, где

- $SN = (P_{SN}, T_{SN}, F_{SN})$ сеть Петри, которая называется системной сетью.
- для каждого $i = \overline{1, k}$, $EN_i = (P_{EN_i}, T_{EN_i}, F_{EN_i})$ – сеть Петри, которая называется элементной сетью, где все множества позиций и переходов не пересекаются; также считается что каждой элементной сети присвоен тип из $Type$.
- $\nu : P_{SN} \rightarrow Type \cup \{\bullet\}$ – функция типизации позиций.
- $\lambda : T_{NP} \rightarrow Lab$ – частичная функция разметки позиций, где $T_{NP} = T_{SN} \cup T_{EN_1} \cup \dots \cup T_{EN_k}$;
- $W : F_{SN} \rightarrow Var \cup \{\bullet\}$ – функция разметки дуг, которая согласована с типами позиций и переменных.

Маркированная элементная сеть называется сетевой фишкой.

Для каждой вложенной сети Петри определяется множество $A_{net} = \{(EN, m) \mid \exists i = 1, \dots, k : EN = EN_i, m \in \mathcal{M}(EN_i)\}$ всех возможных сетевых фишек, и $A = A_{net} \cup \{\bullet\}$.

Маркировка M во вложенной сети Петри NP – это функция, которая сопоставляет каждому $p \in P_{SN}$ некое мультимножество $M(p)$ над A .

Пусть t – переход в SN , $\bullet t = \{p_1, \dots, p_i\}$, $t^\bullet = \{q_1, \dots, q_j\}$. Тогда,

$$W(t) = \{W(p_1, t), \dots, W(p_i, t), W(t, q_1), \dots, W(t, q_j)\}$$

– множество переменных на дугах, смежных с t . Связывание для t – это функция b , которая присваивает каждой переменной из $W(t)$ значение из A .

Переход t называют активным в маркировке M со связыванием b , если $\forall p \in \bullet t : W(p, t)(b) \subseteq M(p)$.

Активный переход может сработать, в результате чего получается новая маркировка M' ($M \rightarrow M'$), такая что $M'(p) = (M(p) \setminus W(p, t)(b)) \cup W(t, p)(b)$.

Существует три типа шагов во вложенных сетях Петри.

- Элементно-автономный шаг. Пусть t – непомеченный переход в одной из системных сетей. Тогда срабатывание перехода t определяется стандартными правилами срабатывания перехода в сетях Петри.
- Системно-автономный шаг. Пусть t – непомеченный переход в системной сети. Срабатывание t определяется правилами срабатывания переходов в сетях высокого уровня (например, см [16]), как описано выше.
- Шаг синхронизации. Пусть t – помеченный λ переход в системной сети, который активен в маркировке M со связыванием b . Пусть $\alpha_1, \dots, \alpha_k \in A_{net}$ – сетевые фишки, участвующие в срабатывании перехода t . Тогда t может сработать если в каждом из α_i существует активный переход k_i , также помеченный λ . При этом, при срабатывании перехода t должны сработать переходы k_i , $0 < i \leq k$.

Вложенная сеть Петри называется *безопасной*, если в каждой ее достижимой маркировке в системной сети в каждой позиции находится не более одной фишки, а также во всех сетевых фишках находится не более одной черной фишки в каждой из позиций. В данной работе рассматриваются только безопасные NP-сети.

В данной работе рассматривается определенный подкласс вложенных сетей Петри – консервативные NP-сети.

Определение 2.2.

NP -сеть $N = SN, (EN_1, \dots, EN_k), \nu, \lambda, W$ называется консервативной, если

1. Для каждого $t \in T_{SN}$ и для каждого $p \in \bullet t$, $\exists! p' \in t^\bullet . W(p, t) = W(t, p')$ или $W(p, t) = \bullet$
2. Для каждого $t \in T_{SN}$ и для каждого $p \in t^\bullet$, $\exists! p' \in \bullet t . W(p', t) = W(t, p)$ или $W(p, t) = \bullet$
3. N безопасна

Данное определение устанавливает, что сетевые фишки в системной сети могут совершать внутренние действия, но новые сетевые фишки не могут появляться, а существующие не могут уничтожаться.

Идентифицированные сетевые фишки. В консервативных вложенных сетях, вместо того что бы рассматривать просто сетевые фишки (маркированные элементные сети находящиеся в позициях системной сети), имеет смысл рассматривать *идентифицированные* сетевые фишки. Так как количество сетевых фишек в сети не меняется, каждой из них можно присвоить идентификатор. Формально, идентифицированная сетевая фишка – это тройка $\langle I, EN, \mu \rangle$, где I является уникальным идентификатором фишки, EN_j представляет из себя структуру фишки (т.е. элементная сеть из множества $\{EN_1, \dots, EN_k\}$), и μ – маркировка EN_j . Представляя фишки таким образом, можно различать сетевые фишки с одинаковой структурой и маркировкой. Это пригодится в главе 3 при определении ветвящихся процессов для консервативных вложенных сетей Петри.

Под $NTok$ будет пониматься множество всех идентифицированных сетевых фишек в данной сети. С помощью $\tau(\eta_i) = \tau((I, EN_i, \mu))$ будет обозначаться тип сетевой фишки (EN_i, μ) . Поскольку далее в тексте будут упоминаться только консервативные вложенные сети Петри, слово “идентифицированные” будет часто опускаться.

По системной сети SN , множеству сетевых фишек $NTok$ и функции $\mathfrak{M}$, сопоставляющей позиции SN элементы из $NTok \cup \{\bullet\}$, можно легко восстановить множество элементных сетей и маркировку вложенной сети. Это является альтернативным заданием вложенной сети Петри, который будет использован в работе.

По маркировке M вложенной сети NP определим *проекции маркировки* на разные компоненты NP :

1. Проекция M на системную сеть SN , обозначается как $M_{\upharpoonright SN}$, это маркировка плоской сети SN , полученная заменой всех сетевых фишек на черные фишки, т.е. $M_{\upharpoonright SN}(p) = |M(p)|$.

2. Проекция M на сетевую фишку $\eta_k = \langle id_k, EN_k, \mu_k \rangle$, обозначается как $M_{\upharpoonright \eta_k}$, это просто μ_k .

Дополнительную информацию по вложенным сетям Петри можно получить в [35].

2.2 Примеры моделирования

В этой секции рассматривается два примера использования вложенных сетей Петри при моделировании мультиагентных систем. Первый пример представляет из себя простой одноранговый протокол, и он является иллюстрационным примером из статьи [7]. Второй пример это инструмент для моделирование беспроводных сенсорных сетей, который описан в [3].

P2P протокол. Данный пример представляет из себя вложенную сеть Петри, моделирующую протокол одноранговой сети (peer-to-peer, p2p).

На рис. 11 можно увидеть системную сеть SN , отвечающую за сам протокол, а так же элементные сети, моделирующие сидов E_1 , предоставляющих данные, пиров E_2 , потребляющие данные, и безопасные каналы передачи E_3 . Сиды и пиры моделируются абстрактно, так как в данной модели описывается только взаимодействие агентов.

Сид E_1 может быть в одном из состояний “готов к загрузке” p_1 , “загрузка в процессе” p_2 или “ре-инициализация” p_3 . В качестве переходов в сиде выступают события “загрузка началась” t_1 , “загрузка закончилась” t_2 и “ре-инициализация закончилась” t_3 . Пир может находиться в состоянии “готов к загрузке” p_4 или “загрузка в процессе” p_5 . Событиями пира являются “загрузка началась” t_4 и “загрузка закончилась” t_5 .

Канал связи (pipe) позволяет улучшать качество загрузки данных за счет буферизации данных. Канал E_3 имеет интерфейсы для сидов (p_6, p_7) и для пиров (p_{12}, p_{13}), а также двухместный буфер (p_8, p_9, p_{10}, p_{11}). Изначально, канал находится в состоянии ожидания (p_{15}), когда сид и пир начинают взаимодействие (t_{11}). Сид может начать (t_6) передавать данные (p_6) в буфер, а пир начать ожидания порции данных. Когда передача данных заканчивается (t_7), сид возвращается в состояние ожидания (p_7), и одно место в буфере заполнено (p_8).

Затем первая порция данных передается (t_8) во второе место буфера, в результате чего первое место буфера становится свободным (p_9). Когда пир загружает (t_9) первую порцию данных из второго места буфера (p_{10}), второе место буфера становится пустым (p_{11}), и пир начинает ожидать второй порции данных (p_{13}).

Когда сид загрузил все свои данные, канал ожидает передачи данных к пиру (p_{13}),

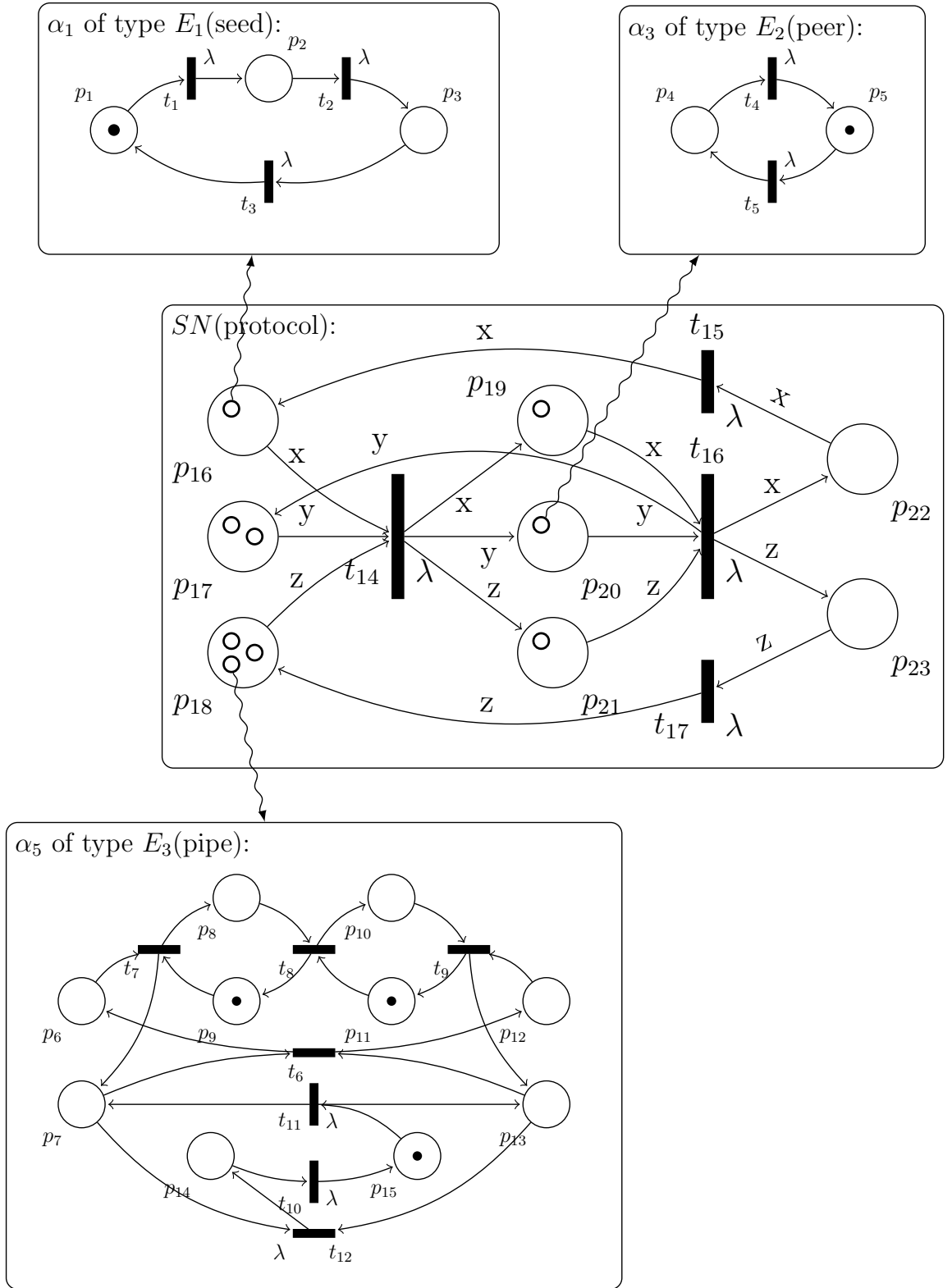


Рис. 11: Вложенная сеть Петри, моделирующая R2P протокол [7]

а затем перемещается (t_{12}) состояние ре-инициализации (p_{14}). Когда ре-инициализация заканчивается (t_{10}), канал завершает цикл трансфера и готов заново принимать и передавать данные (p_{15}).

Системная сеть содержит начальные и конечные пулы для сидов, пиров и каналов

(p_{16} , p_{17} и p_{18} соответственно). В начальном пуле агенты готовы к взаимодействию, а в конечном пуле агенты проходят ре-инициализацию.

Данная модель является высокоуровневым представлением протокола, и в ней не учитываются такие вещи как чрезвычайны ситуации (например сбой канала) и их обработка.

Моделирование беспроводных сенсорных сетей. В работе [3] описывается инструмент для моделирования беспроводных сенсорных сетей на базе вложенных сетей Петри.

Беспроводные сенсорные сети (Wireless sensor networks, WSN) – это распределенные компьютерные сети, состоящие из беспроводных сенсоров, которые считывают какую-либо информацию, обмениваются ей друг с другом и передают ее на хост (сервер). Часто сенсоры используются для мониторинга физических условий окружающей среды, таких как температура, атмосферное давление и пр. Одним из применений беспроводных сенсорных сетей является наблюдение за дикими животными. Ученые, с помощью сенсорных сетей с камерами пытаются определить тропы животных.

Описываемый в статье инструмент позволяет пользователю задавать беспроводную сенсорную сеть, а затем конвертировать ее во вложенную сеть Петри, с целью анализа.

Конвертация сенсорной сети в NP-сеть происходит следующим образом. Сначала, каждому сенсорному устройству сопоставляется по две позиции, соединенные переходом, как можно увидеть на рис. 12. В одной из позиций находится слетевшая фишка, описывающее поведение сенсора, в другой – может находиться обычная фишка, обозначающая сигнал. Затем, сенсоры, находящиеся рядом, соединены переходами. Поведение отдельных сенсоров задается пользователем в виде элементных сетей.

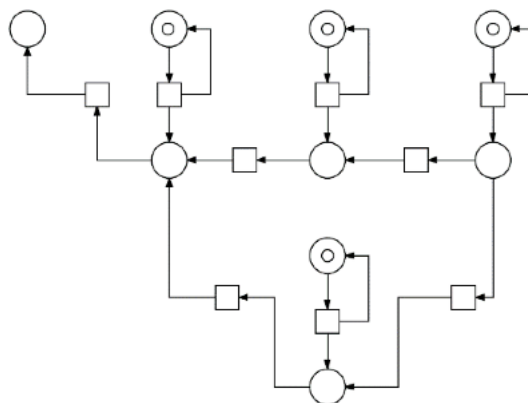


Рис. 12: Вложенная сеть Петри, моделирующая WSN [3]

Применение NP-сетей. В данной главе были представлены два примера применения формализма вложенных сетей Петри к системам из разных сфер.

Эти примеры показывают, что вложенные сети Петри являются удобным формализмом для моделирования мультиагентных системы. Если бы мы моделировали такие системы с помощью плоских сетей, то полученные модели были бы очень громоздкими. За счет того, что вложенные сети Петри позволяют задавать поведение агентов отдельно от протокола общего поведения, полученные модели являются обозримыми.

Эти примеры хорошо подходят для моделирования с помощью формализма с жесткой иерархией и value-семантикой. Одно поведение (одну сеть) задается нескольким агентам сразу. Такое было бы невозможно сделать в формализмах гиперсетей [26] или TSA-систем [32].

2.3 Библиографический комментарий

Одной из ранних разработок в области “nets within nets” является формализм Объектных сетей, который позже был сужен до Элементарных Объектных Сетей Петри (Elementary Object Nets, EON) [33], основанный в определенной степени на идеях объектно-ориентированного программирования. Сети, используемые как фишки, в EON называются *сетевыми фишками*, и они имеют свою собственную структуру и поведение.

Другой формализм, основывающийся на парадигме “сети-внутри-сетей” – это Гиперсети [26, 1, 25]. Гиперсеть – это набор агентов, вместе с *гипермаркировкой* – распределением агентов по позициям сети, каждый из которых является произведением конечных автоматов (или консервативной сетью Петри, в случае обобщенных гиперсетей). Каждый агент может находиться в любом из других агентов (за исключением циркулярных ситуаций), и такая иерархия может меняться во время исполнения.

Формализм, который рассматривается в этой работе – это вложенные сети Петри [22, 13, 24, 35]. Этот формализм был выбран для изучения по нескольким причинам: во-первых, это удобство для моделирования большого класса систем; во-вторых, вложенные сети Петри являются не Тьюринг-полными, но более выразительными чем обычные сети Петри, что делает некоторое количество проблем разрешимыми (например, проблему покрытия [35]). Вложенные сети Петри отличаются от других формализмов “сети-внутри-сетей” и другими свойствами. Например, в отличие от uTSA-систем ([32]), вложенные сети Петри имеют т.н. value semantics, что означает что взаимодействие с агентами (сетевыми фишками) происходит локально путем прямого взаимодействия с фишками, а не глобально через специальные идентификаторы. Также, в отличие от Гиперсетей, вложенные сети Петри имеют статическую иерархию, которая не может изменяться во время исполнения сети.

3 Развертки вложенных сетей Петри

В этой главе будут рассмотрены два варианта определения разверток для консервативных вложенных сетей Петри: покомпонентная развертка, в которой каждый компонент разворачивается отдельно, но остается компонентом вложенной сети, и прямая развертка вложенной сети Петри в сеть событий. Как можно будет увидеть, второй вариант является более привлекательным, поскольку именно он позволяет применить метод разверток для верификации вложенных сетей Петри.

3.1 Покомпонентная развертка сетей

Поскольку в консервативных NP-сетях сетевые фишки не создаются и не уничтожаются, естественным будет рассмотреть отдельные развертки всех индивидуальных компонент сети и их взаимодействия. Композициональность такой конструкции является интересной и может сыграть положительную роль при верификации. Такая “развертка” (далее в тексте называемая “покомпонентной разверткой”, *component-wise unfolding*), безусловно, должна иметь свойство, схожее с фундаментальной теоремой разверток (свойство 1.1).

Покомпонентная развертка NP-сети $N = (SN, NToK = \{\eta_1, \dots, \eta_k\}, \nu, \lambda, W, M^0)$, это NP-сеть $U_C(N) = (U_C(SN), U_C(NToK), U_C(\lambda), U_C(W), U_C(M^0))$, полученная разверткой каждого компонента сети в отдельности:

- $U_C(SN)$ – это развертка плоской сети SN с начальной маркировкой $M^0_{|SN}$;
- $U_C(NToK)$ – это множество сетевых фишек, каждая из которых развернута по отдельности.

Пусть $h : P_{U_C(SN)} \cup T_{U_C(SN)} \cup_{0 < i \leq k} P_{U_C(\eta_i)} \cup P_{U_C(\eta_i)} \rightarrow P_{SN} \cup T_{SN} \cup_{0 < i \leq k} P_{\eta_i} \cup P_{\eta_i}$ – объединение всех морфизмов из разверток в индивидуальные компоненты. Т.е.

$$h(x) = \begin{cases} h_{SN}(x), & \text{если } x \in P_{U_C(SN)} \cup T_{U_C(SN)} \\ h_{\eta_i}(x), & \text{если } x \in P_{U_C(\eta_i)} \cup T_{U_C(\eta_i)} \end{cases}$$

Тогда функции $U_C(W)$, $U_C(\lambda)$ и $U_C(M^0)$ определяются следующим образом:

- $U_C(W)(x, y) = W(h(x), h(y))$;
- $U_C(\lambda)(t) = \lambda(h(t))$.

- $U_C(M^0)(p) = M^0(h(p))$ если p минимальный элемент (с точки зрения $<$) в множестве $\{p' \mid h(p') = h(p)\}$, или $U_C(M^0)(p) = \emptyset$ в противном случае.

Прежде чем исследовать поведенческие свойства такой развертки, требуется расширить определение функции h (морфизма), что бы оно включало также NP-маркировки:

Пусть M_U – маркировка $U_C(NP)$, тогда соответствующая ей маркировка $h(M_U)$ определяется следующим образом:

$$h(M_U)(p) = \begin{cases} h(S), & \text{если } \exists p'. h(p') = p \wedge M_U(p') = S \neq \emptyset \\ \emptyset, & \text{в противном случае} \end{cases} \quad (3.1)$$

Важно заметить, что функция определенная в уравнении 3.1 является корректной: если выполняется первое условие уравнение, то такое p' уникально, ведь никакие два элемента из множества $\{p' \mid h(p') = x\}$ не параллельны и, следовательно, не могут быть помечены ни в какой достижимой маркировке одновременно.

Теперь все готово для формулирования и доказательства теоремы о соответствии поведения NP-сети и ее покомпонентной развертки.

Теорема 3.1.

Пусть NP – консервативная NP-сеть. Пусть M – достижимая маркировка NP , а M_U – достижимая маркировка $U_C(NP)$, такая что $h(M_U) = M$. Тогда выполняется следующее:

1. *Если существует шаг $M_U \xrightarrow{T_U} M'_U$ в $U_C(NP)$, то существует шаг $M \xrightarrow{T} M'$ в NP , такой что $h(T_U) = T$ и $h(M'_U) = M'$.*
2. *Если существует шаг $M \xrightarrow{T} M'$ в NP , то существует шаг $M_U \xrightarrow{T_U} M'_U$ в $U_C(NP)$, такой что $h(T_U) = T$ и $h(M'_U) = M'$.*

То есть поведение сети NP эквивалентно поведению ее покомпонентной развертки.

Доказательство. Рассмотрим первое утверждение и шаг синхронизации. Остальные случаи рассматриваются подобным образом.

Для каждого из компонент $U_C(N_i), i \in I_T$, занятого в срабатывании T_U , существует переход $t_i \in T_U$. Также, существует $t_s \in T_U$, принадлежащий системной сети.

Для каждого t_i существует позиция p_i в системной сети где находится фишка $U_C(N_i)$. В таком случае, $\bullet t_i \subseteq M_U(p_i)$. Более того, если срабатывание T переносит фишку $U_C(N_i)$ из p_i в позицию p'_i , то $M'_U(p'_i) = M_U(p_i) \setminus \bullet t_i \cup t_i \bullet$. Это значит, что в системной фишке $U_C(N_i)$ (как

в отдельном компоненте) $M_U(p_i) \xrightarrow{t_i} M'_U(p'_i)$. Следовательно, по фундаментальной теореме разверток для плоских сетей, $h(M_U(p_i)) = M(h(p_i)) \xrightarrow{h(t_i)} M'(h(p'_i)) = h(M'_U(p'_i))$ в N_i .

Также можно заметить, что существует шаг $M_U \xrightarrow{t_S} M'_U$ в $U_C(SN)$. Следовательно существует шаг $h(M_U \upharpoonright_{SN}) \xrightarrow{h(t_S)} h(M'_U \upharpoonright_{SN})$ в SN . Вместе с тем фактом, что U_C сохраняет метки на переходах, получаем что переход $T = h(T_U)$ активен в маркировке M сети NP . Нетрудно заметить что M' совпадает с $h(M'_U)$. $\square$

Не смотря на то, что вышеописанная конструкция имеет ряд привлекательных теоретических свойств, ее прямая применимость к процессу композициональной проверки моделей не вполне очевидна. Более точно, не вполне понятно, каким образом анализ разверток индивидуальных компонент может помочь анализу развертки всей сети целиком. Так, например, не-живость хотя бы одной из компонент сети (свойство живости можно проверить на развертках индивидуальных компонент) ведет к не-живости всей сети целиком, однако живость всех компонент сети не обязательно приводит к живости сети в целом¹.

В следующей секции будет рассмотрен другой подход к определению разверток NP-сетей, в котором поведение NP-сети представляется на прямую одной общей сетью событий.

3.2 Ветвящиеся процессы вложенных сетей Петри

В этой части будет рассмотрены развертки консервативных вложенных сетей Петри непосредственно в сети событий. Определение развертки будет дано через понятие ветвящегося процесса.

Для удобства зафиксируем на протяжении всей главы консервативную вложенную сеть Петри $NP = (SN, (EN_1, \dots, EN_k), \nu, \lambda, M^0)$, где $SN = (P_{SN}, T_{SN}, F_{SN})$, $EN_i = (P_{EN_i}, T_{EN_i}, F_{EN_i})$, $0 < i \leq k$.

Для начала определим *элементно-индексированные C-сети Петри*. Данная конструкция похожа на каноническое представление ветвящихся процессов плоских сетей Петри, однако в случае элементно-индексированных C-сетей Петри, каждая позиция которой помечена сетевой фишкой (ее идентификатором).

Определение 3.1 (Элементно-индексированные C-сети).

Элементно-индексированная C-сеть Θ – это C-сеть, в которой каждая позиция помечена идентификатором сетевой фишки.

¹Данный вопрос был изучен в [7]

Формально, множество канонических имен C определяется рекурсивно следующим образом:

- Если $x \in (\bigcup_{EN_i} P_{EN_i} \cup P_{SN})$, $\eta_i \in NTok \cup \{\bullet\}$, и X является конечным подмножеством C , то $(X, x, \eta_i) \in C$;
- Если $x \in (\bigcup_{EN_i} T_{EN_i} \cup T_{SN})$, и X – конечное подмножество C , то $(X, x) \in C$.

Тогда индексированная C -сеть (P, T, F, M_0) – это сеть Петри, где

1. $P \cup T \subseteq C$;
2. Если $p = (X, x, N) \in P$, то $\bullet p = X$;
3. Если $t = (X, x) \in T$, то $\bullet t = X$;
4. $(X, x) \in M_0$ тогда и только тогда, когда $X = \emptyset$.

Так же как и в случае простых C -сетей (ф-л. (1.1)), определяется функция h , сопоставляющая вершинам сети событий вершины вложенной сети:

$$h(x) = \begin{cases} t & \text{если } x = (A, t) \\ p & \text{если } x = (A, p, \eta_i) \end{cases} \quad (3.2)$$

Объединение элементарно-индексированных C -сетей определяется так же как и объединение простых C -сетей – покомпонентно.

Вообще говоря, позиция (A, p, i) соответствует либо позиции p в сетевой фишке с идентификатором i , либо позиции p в системной сети, в которой находится сетевая фишка с идентификатором i . Графически, позиция (A, p, i) обозначается как обычная позиция с именем (p, i) .

Для каждой элементарно-индексированной C -сети Θ и сетевой фишки η_i определено множество $Path(\Theta, \eta_i)$ позиций, которые соответствуют позициям в системной сети, помеченные η_i , и смежные к ним переходы. То есть $Path(\Theta, \eta_i)$ содержит все (и только их) позиции вида (A, p, η_i) , а так же переходы $\bullet(A, p, \eta_i)$. Также, $Path(\Theta, S, \eta_i)$ будет обозначать $Path(\Theta, \eta_i) \cap S$. Эта функции называется $Path$, поскольку в случае ветвящихся процессов, $Path(\Theta, \eta_i)$ обозначает “путь” фишки η_i в сети Θ . Так, на рис. 16 все элементы $Path(BP_1, N_2)$ обозначены синим цветом.

Также, для каждого множество S вершин элементарно-индексированной C -сети Θ определено мультимножество пометок:

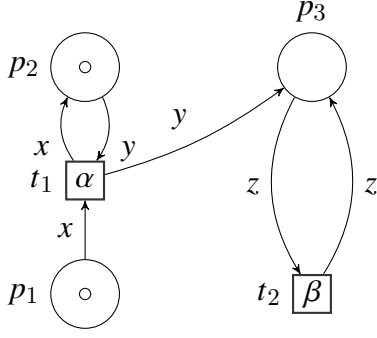


Рис. 13: Сеть NP_2 : системная сеть

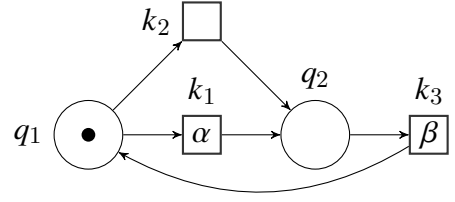


Рис. 14: Сеть NP_2 : Элементная сеть

$$Labs(S) = \{\lambda(t) \mid (A, t) \in S\}$$

Возвращаясь к предыдущему примеру, $Labs(Path(BP_1, N_2)) = \{\alpha\}$.

Также следует определить понятие *смежного места*. В соответствии с определением 2.2, для каждой пары $(p, t) \in P_{SN} \times T_{SN}$, где $v(p) \neq \bullet$, существует уникальная позиция p' , такая что $W(p, t) = W(t, p')$ или $W(p', t) = W(t, p)$. Такая позиция p' называется *смежной к p через t* и обозначается как $\langle \widetilde{p}, t \rangle$. Например, на рис. 13, смежной к p_2 через t_1 является позиция $\langle \widetilde{p_2}, t_1 \rangle = p_3$.

Теперь все готово для определения элементарно-индексированных ветвящихся процессов (далее просто “ветвящихся процессов”, где это позволительно использовать без потери смысла).

Определение 3.2 (Элементарно-индексированные ветвящиеся процессы).

Множество элементарно-индексированных ветвящихся процессов для NP – это наименьшее множество элементарно-индексированных C -сетей, которое удовлетворяет следующим требованиям:

1. Пусть

$$C = \{(\emptyset, p, \eta_i) \mid p \in P_{SN}, \eta_i \in M^0(p)\} \cup \{(\emptyset, p, \eta_i) \mid \eta_i \in NTok, p \in M^0_{\eta_i}\}$$

множество позиций. Тогда сеть $\Theta = (C, \emptyset, \emptyset)$, состоящая из позиций C и не имеющая никаких переходов, является ветвящимся процессом. Такой процесс называют начальным.

2. Пусть Θ – ветвящийся процесс, а B – множество позиций в Θ . Если B удовлетворяет правилу $PosEN$ (рис. 15), то тогда сеть, полученная путем добавления перехода e и позиций C к Θ , является ветвящимся процессом.

3. Пусть Θ – ветвящийся процесс, а B – множество позиций в Θ . Если B удовлетворяет правилу *PosSN* (рис. 15), то тогда сеть, полученная путем добавления перехода e и позиций C к Θ , является ветвящимся процессом.
4. Пусть Θ – ветвящийся процесс, а B и B_E – множество позиций в Θ . Если B и B_E удовлетворяют правилу *PosSync* (рис. 15), то тогда сеть, полученная путем добавления перехода e и позиций C к Θ , является ветвящимся процессом.
5. Пусть $\mathcal{BS}$ является (конечным или бесконечным) множеством ветвящихся процессов. Тогда объединение $\bigcup \mathcal{BS}$ является ветвящимся процессом.

В правилах (2)-(4), переход e называют возможным расширением Θ .

Предикат $\text{SyncCond}(B_E, E, I, \Theta, (B, t))$ является истинным при тогда и только тогда, когда выполняются следующие условия

1. $B_E = \bigcup_{i \in I} \{(y_j, c_j, \eta_i) \mid j \in J_i, \eta_i = (id_i, EN_i, \mu_i) \in NTok, c_i \in P_{EN_i}\} \wedge co(B_E)$, т.е. B_E – множество достижимых условий, соответствующих позициям в сетевых фишках;
2. $E = \{t_i \in T_{EN_i} \mid i \in I, (id_i, EN_i, \mu_i) \in NTok\}$, т.е. E – подмножество переходов в каждой из сетевых фишек;
3. $\forall e \in E, \lambda(e) = \lambda(t)$
4. $\bullet E = \bigcup_{i \in I} \{c_j \mid j \in J_i, \eta_i \in NTok\}$
5. $\forall i \in I, \eta_i \in NTok \rightarrow \text{Labs}(\text{Path}(\Theta, [B], \eta_i)) = \text{Labs}(\{ \{(y_j, c_j, \eta_i) \mid j \in J_i \} \})$, т.е. список пометок на пути η_i в конфигурации $[B]$ такой же, как список пометок в конфигурации $[c_i]$.

Правила, обозначенные в рис. 15, можно понимать так же и с операционной точки зрения: правила *PosEN*, *PosSN* и *PosSync* применяются для получения событий (переходов), соответствующие элементарно-автономным, системно-автономным и синхронизирующим шагам, соответственно.

Для примера, рассмотрим NP-сеть NP_2 , изображенную на рис. 13 и 14. В этом примере в системной сети (рис. 13) находятся две системные фишки одного типа (рис. 14). При срабатывании перехода t_1 системная фишка из позиции p_1 переходит в позицию p_2 , а фишка из позиции p_2 переходит в позицию p_3 . Возможные ветвящийся процессы NP_2 изображены на рис. 16 и 17.

$$\frac{B = \{(x_i, b_i, \eta_k) \mid i \in I\} \quad co(B) \quad t \in T_{\eta_k}, \lambda(t) = * \quad \bullet t = \{b_i \mid i \in I\}}{e = (B, \{t\}) \quad \text{and} \quad C = \bigcup_{p \in \bullet} (e, p, \eta_k)} PosEN$$

$$\frac{B = \{(x_i, b_i, \eta_i) \mid i \in I\} \quad co(B) \quad t \in T_{SN}, \lambda(t) = * \quad \bullet t = \{b_i \mid i \in I\}}{e = (B, \{t\}) \quad \text{and} \quad C = \{(e, \langle \widetilde{b_i}, t \rangle, \eta_i) \mid i \in I, \eta_i \neq \bullet\} \cup \{(e, b, \bullet) \mid b \in t^\bullet, v(b) = \bullet\}} PosSN$$

$$\frac{B = \{(x_i, b_i, \eta_i) \mid i \in I\} \quad t \in T_{SN}, \lambda(t) \neq * \quad co(B) \quad \bullet t = \{b_i \mid i \in I\} \quad SyncCond(B_E, E, I, \Theta, (B, t))}{e = (B \cup B_E, \{t\} \cup E) \quad \text{and} \quad C = \{(e, \langle \widetilde{b_i}, t \rangle, \eta_i) \mid i \in I, \eta_i \neq \bullet\} \cup \{(e, b, \bullet) \mid b \in t^\bullet, v(b) = \bullet\} \cup \{(t', c'_i, \eta_i) \mid i \in I, \eta_i \neq \bullet, c'_i \in P_{\eta_i}, c'_i \in t_i^\bullet\}} PosSync$$

Рис. 15: Правила для построения возможных расширений ветвящихся процессов

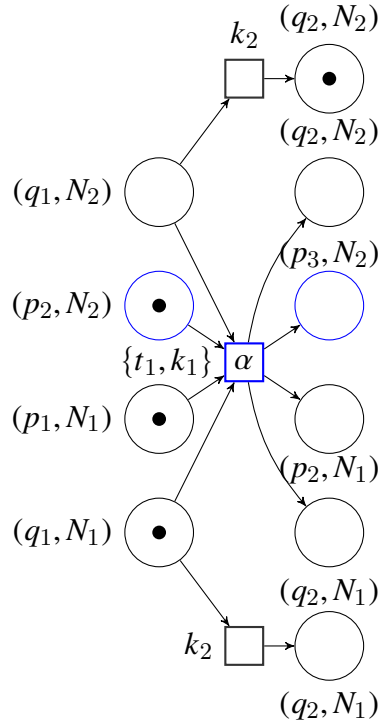


Рис. 16: BP_1 : ветвящийся процесс NP_2

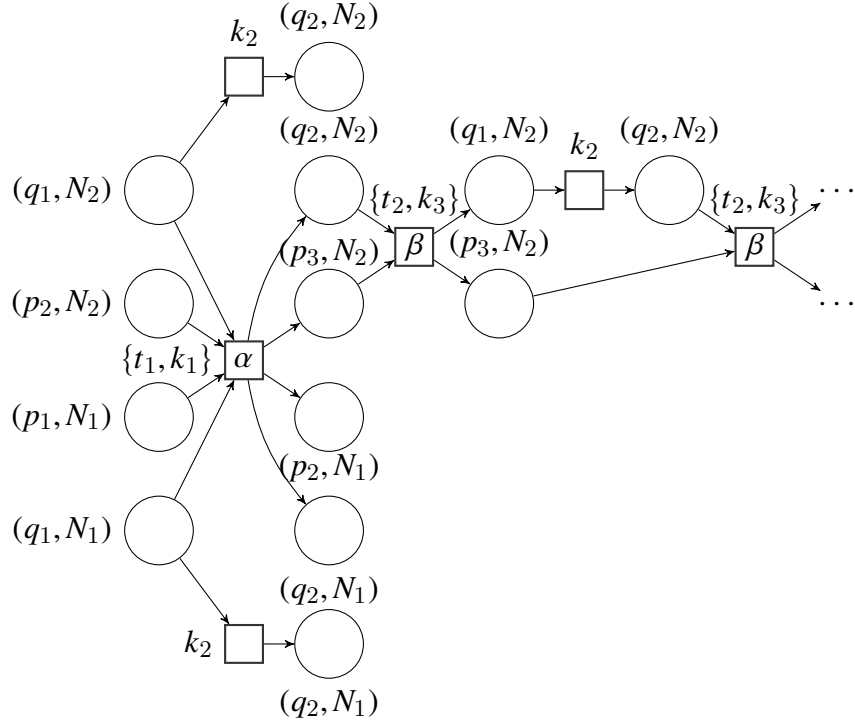


Рис. 17: BP_2 : ветвящийся процесс NP_2

Свойства элементарно-индексированных ветвящихся процессов. Далее будут сформулированы и доказаны важные свойства, касающиеся ветвящихся процессов, как они были определены.

Утверждение 3.1.

Каждый элементарно-индексный ветвящийся процесс является сетью событий.

Доказательство. Пусть $\Theta = (P_\Theta, T_\Theta, F_\Theta)$ – элементарно-индексированный ветвящийся процесс. Что бы доказать что Θ является сетью событий, надо удостовериться что для Θ выполняются все условия, указанные в определении 1.2.

1. Это условие выполняется потому что Θ является элементарно-индексированной S -сетью Петри.
2. Можно удостовериться что это условие выполняется, посмотрев на каждой из индуктивных правил и заметив, что при применении каждого из правил каждая новая позиция имеет лишь один пре-переход.
3. Выполнимость этого условия следует прямо из индуктивных правил возможных расширений.
4. Докажем выполнимость этого условия от обратного. Пусть такая наименьшая (с точки зрения $<$) вершина $x \in P_\Theta \cup T_\Theta$, что $x \# x$. Тогда $\exists t, t' \in T_\Theta$. $t < x \wedge t' <$

$x \wedge \bullet t \cap \bullet t \neq \emptyset$ (по определению отношения конфликта). Если x является позицией, то, по пункту 2, существует единственный переход y , такой что $\bullet x = \{y\}$ и $y \# x$ (x не может не иметь пред-переходов, так как в соответствии с гипотезой $t < x$ и $t' < x$). Более того, $x \# x \iff y \# y$. Поэтому без ограничения общности, достаточно рассмотреть случай когда x – переход.

Если x – переход, то он имеет вид (B, t) , и существуют такие различные $b, b' \in B$, что $t < b, t' < b'$. В таком случае $b \# b'$, т.е. $\neg co(B)$. Однако, такая ситуация невозможна, в соответствии с правилами возможных расширений.

□

Пусть B – ветвящийся процесс NP . Следующим шагом мы определим функцию $\tilde{h}$, расширяющую функцию h из формулы 3.2, сопоставляющую каждой маркировке ветвящегося процесса B маркировку NP . В случае плоских сетей специального расширения определения морфизма не требовалось, так как маркировкой плоской сети Петри является мультимножество позиций, и образ мультимножества позиций относительно функции h определяется стандартным образом. В случае вложенных сетей, маркировки представляют из себя функции из позиций системной сети в мультимножества сетевых фишек. Можно заметить, что в случае когда в сети отсутствуют сетевые фишки, определение совпадает просто с функцией h .

$$\tilde{h}(M)(p) = \begin{cases} \{\bullet\} & \text{если } v(p) = \bullet \text{ и } h(M)(p) > 0 \\ \{\{q \mid q \in P_{\eta_i}, \exists A, (A, q, \eta_i) \in M\}\} & \text{если } v(p) = \tau(\eta_i) \text{ и } \exists A, (A, p, \eta_i) \in M \\ \emptyset & \text{в противном случае} \end{cases} \quad (3.3)$$

Допустим, M – маркировка BP_1 , изображенная на рис. 16. Тогда $\tilde{h}(M)$ – маркировка NP_2 , изображенная на рис. 18.

Такое определение корректно, поскольку для каждой сетевой фишки η_k в каждой достижимой маркировке B , лишь только одна позиция может быть помечена η_k . Поскольку каждый ветвящийся процесс является сетью событий, то каждая достижимая маркировка является попарно-параллельным множеством. Следовательно, требуется доказать следующую лемму:

Лемма 3.1.

Пусть η_k является системной фишкой, а θ – ветвящимся процессом. Если $p_i, p_j \in P_{SN}$, $p_i \neq p_j$ и $(A, p_i, \eta_k), (B, p_j, \eta_k) \in \theta$, то $\neg((A, p_i, \eta_k) co (B, p_j, \eta_k))$.

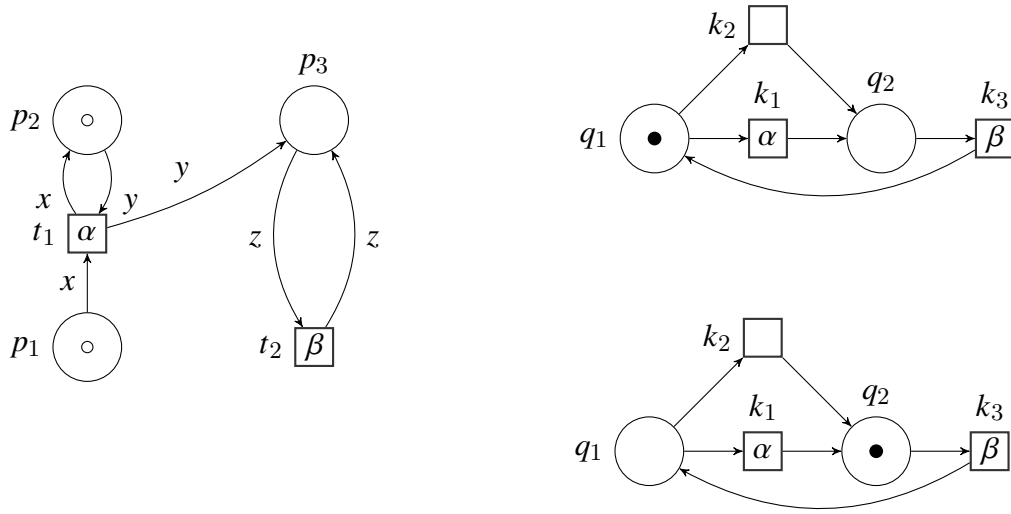


Рис. 18: NP-сеть NP_2 с маркировкой $\tilde{h}(M)$ (рис. 16). Слева направо, сверху вниз: системная сеть SN , находящаяся в p_1 системная фишка N_1 , находящаяся а p_2 системная фишка N_2 .

Доказательство. Доказательство проводится при помощи структурной индукции на определении ветвящегося процесса. Утверждение леммы очевидно выполняется для начального ветвящегося процесса.

Пусть утверждение выполняется для θ , а θ' – расширение θ , полученной применением одного из правил возможных расширений. Пусть C – множество новых добавленных позиций, помеченных позициями системной сети. Можно заметить, что все элементы C помечены разными сетевыми фишками. Следовательно, для доказательства леммы требуется показать что $\forall (A, p_i, \eta_k) \in \theta, \forall (B, p_j, \eta_k) \in C, \neg((A, p_i, \eta_k) \text{ со } (B, p_j, \eta_k))$.

Правило *PosEN* не вводит новых позиций, помеченных позициями системной сети; в таком случае множество C пустое. В случае с правилом *PosSN* можно заметить что существует такая позиция (A', p_l, η_k) в θ , что $(A', p_l, \eta_k) \in \bullet\bullet(B, p_j, \eta_k)$, где p_j является смежным к p_l , для каждого (B, p_j, η_k) из C . По индуктивной гипотезе, $\neg((A', p_l, \eta_k) \text{ со } (A, p_i, \eta_k))$. Следовательно, $\neg((A, p_i, \eta_k) \text{ со } (B, p_j, \eta_k))$.

Случай *PosSync* может быть рассмотрен в таком же виде. □

Поскольку плоские сети Петри являются частным случаем вложенных сетей Петри (без элементных сетей и синхронизации), то можно сформулировать и доказать следующее свойство, которое является в своем роде “sanity check”, проверяющим то что определение элементарно-индексированных ветвящихся процессов определено верно для плоских сетей Петри.

Утверждение 3.2.

Пусть N – безопасная сеть Петри. Множество ветвящихся процессов N изоморфно множеству элементарно-индексированных ветвящихся процессов N , рассмотренной как вложенная сеть Петри.

Доказательство. Единственное возможное правило, которое может быть применено при построении элементарно-индексированного ветвящегося процесса, в данном случае является $PosSN$. Можно заметить, что в случае отсутствия сетевых фишек, правило $PosSN$ совпадает с правилом возможных расширений для ветвящихся процессов сетей низкого уровня. $\square$

Таким образом, можно расширить определение префиксного отношения $\sqsubseteq$, охватывая ветвящиеся процессы вложенных сетей Петри. Развертка (обозначается $U(NP)$) вложенной сети Петри NP в таком случае определяется как $\sqsubseteq$ -максимальный ветвящийся процесс NP .

Благодаря этому можно сформулировать фундаментальное свойство разверток, которое выполняется для вложенных сетей Петри.

Теорема 3.2.

Пусть M – достижимая маркировка NP и M_U – достижимая маркировка $U(NP)$, такая что $\tilde{h}(M_U) = M$

1. Если $M_U \xrightarrow{t_U} M'_U$ в $U(NP)$, то существует шаг $M \xrightarrow{t} M'$ в NP , такой что $\tilde{h}(M'_U) = M'$ и $h(t_U) = t$.
2. Если $M \xrightarrow{t} M'$ в NP , то существует шаг $M_U \xrightarrow{t_U} M'_U$ в $U(NP)$, такой что $\tilde{h}(M'_U) = M'$ и $h(t_U) = t$.

В следующей главе будет рассмотрено применение базового алгоритма для построения конечных префиксов разверток вложенных сетей Петри на базе теории контекста среза-ния, а также применение конечных префиксов вложенных сетей Петри для верификации поведенческих свойств NP-сетей.

4 Верификация с помощью разверток

В данной главе будет рассмотрено применение разверток сетей для верификации различных свойств вложенных сетей Петри. Основная идея такого подхода к верификации заключается в использовании конечных префиксов разверток, которые не представляют поведение оригинальной сети полностью, насколько его представляет сама развертка, но которые сохраняют необходимые для верификации свойства. В начале главы будут представлены необходимые определения и небольшая теоретическая база. Далее будут рассмотрены алгоритмы, позволяющие строить необходимые конечные префиксы. Материал этой главы основан на исследованиях теории канонических префиксов [19, 17].

В случаях когда система имеет бесконечные исполнения развертка является неограниченной и, следовательно, не может применяться для верификации. Для анализа поведенческих свойств таким образом используются конечные префиксы развертки, которые сохраняют достаточную информацию для корректного анализа оригинальной системы. Основной характеристикой существующих алгоритмов генерация конечных префиксов является концепция событий-отсечек (cut-off events), после которых информация в развертке начинает повторяться. Именно после таких событий развертку можно сокращать до префикса. В теории канонических префиксов понятие событие-отсечки является независимым от алгоритма и, следовательно, применимо почти к любому существующему алгоритму. Какое событие является отсечкой, а какое – нет, определяется по контексту срезания, которые определяет какое количество информации оставить в префиксе развертки. В данном тексте в качестве базового алгоритма рассмотрен алгоритм ERV [11], а точнее его обобщение с использованием теории канонических префиксов и контекстов срезания.

Благодаря результатам предыдущей главы, вся теория о канонических префиксах может применяться к консервативным вложенным сетям Петри практически без изменений.

4.1 Канонические префиксы разверток

Для начала, вспомним определение конфигурации (1.3): бесконфликтное множество вершин, замкнутое относительно $<$. Для ветвящегося процесса B множества конечных и локальных конфигураций обозначаются $\mathfrak{C}_{fin}^B$ и $\mathfrak{C}_{loc}^B$. Верхний индекс опускается, если B является разверткой.

Для каждой конфигурации C определяется *срез*:

$$Cut(C) = (\bigcup_{e \in C} e^\bullet) \setminus (\bigcup_{e \in C} \bullet e)$$

Срез определяет достижимое состояние ветвящегося процесса, и, соответственно, достижимое состояние оригинальной сети Петри: $Mark(C) = \tilde{h}(Cut(C))$.

Главным понятием при определении канонических префиксов является *контекст срезания* (*cutting context*).

Определение 4.1.

Контекст срезания (*cutting context*) – это тройка $\kappa = (\approx, \triangleleft, \{\mathfrak{C}_e\}_{e \in E})$, где

1. $\approx$ – отношение эквивалентности на $\mathfrak{C}_{fin}$.
2. $\triangleleft$ – строгий, фундированный порядок на $\mathfrak{C}_{fin}$, улучшающий $\subset$ (т.е. если $C \subset C'$, то $C \triangleleft C'$). Такой порядок называется адекватным.
3. $\approx$ и $\triangleleft$ сохраняют конечные расширения, т.е. если $C', C'' \in \mathfrak{C}_{fin}$, $C' \approx C''$ и E' – конечный суффикс C' , то существует такой суффикс E'' , что
 - $C' \oplus E' \approx C'' \oplus E''$
 - Если $C'' \triangleleft C'$, то $C'' \oplus E'' \triangleleft C' \oplus E'$
4. $\{\mathfrak{C}_e\}_{e \in E}$ – семейство конфигураций для каждого перехода из развертки.

Условно говоря, $\approx$ определяет уровень информации, который должен оставаться в каноническом префиксе: в префиксе должны присутствовать элементы из каждого класса эквивалентности, заданного $\approx$. В изначальном алгоритме МакМиллана две конфигурации $C' \approx C''$ если они представляли собой одни и те же маркировки $Mark(C') = Mark(C'')$. Однако, в некоторых случаях такая информация может быть избыточной. Например, нас может интересовать равенство маркировок с точностью до симметрии, или равенство маркировок какой-либо элементной сети.

Отношение $\triangleleft$ на конфигурации имеет техническое значение: именно в этом порядке будут выбираться кандидаты для событий-отсечек (cut-off событий). В результате, все $\triangleleft$ -минимальные конфигурации в каждом из классов эквивалентности $\approx$ будут сохранены. МакМиллан в качестве адекватного порядка использовал порядок размера: $C' \triangleleft C'' \iff |C'| < |C''|$.

Последний параметр контекста срезания определяет конфигурации, которые будут рассматриваться для поиска событий-отсечек. Обычно рассматриваются только локальные конфигурации, и в качестве этого параметра выступает $\{\mathfrak{C}_{loc}\}_{e \in E}$.

Для простоты, пишут $e \triangleleft e'$ если $[e] \triangleleft [e']$.

Далее определяется понятие *полноты префикса*.

Определение 4.2.

Ветвящийся B называется *полным по отношению к множеству событий-отсечек* (cut-off событий) E_{cut} , если:

1. Для каждого $C \in \mathfrak{C}_{fin}$ существует $C' \in \mathfrak{C}_{fin}^B$, такой что $C' \cap E_{cut} = \emptyset$ и $C \approx C'$;
2. Если $C \in \mathfrak{C}_{fin}^B$ такой что $C \cap E_{cut} = \emptyset$, а e – событие, такое что $C \oplus \{e\} \in \mathfrak{C}_{fin}$, то $C \oplus \{e\} \in \mathfrak{C}_{fin}^B$.

Ветвящийся процесс называется *полным*, если он полный по отношению к какому-нибудь множеству событий-отсечек.

Первое условие говорит о том, что все классы эквивалентности $\approx$ должны быть представлены в полном ветвящемся процессе. Второе условие утверждает что все не отрезанные конечные конфигурации в полном ветвящемся процессе должны быть продолжены, если такое возможно.

Далее приводится алгоритмо-независимое определение событий-отсечек (cut-off событий).

Определение 4.3.

Для заданного контекста срезания k множество возможных (feasible) событий, обозначаемое $fsble_k$, и множество событий-отсечек, обозначаемое cut_k , определяются следующим образом:

- Событие e принадлежит $fsble_k$, если $([e] \setminus \{e\}) \cap cut_k = \emptyset$.
- Событие e принадлежит cut_k , если $e \in fsble_k$ и существует такая конфигурация $C \in \mathfrak{C}_e$ что $C \subseteq fsble_k \setminus cut_k$, $C \approx [e]$, $C \triangleleft [e]$. Конфигурация C удовлетворяющая таким условиям называется соответствующей событию e .

Поскольку возможные (feasible) события и события-отсечки (cut-off) определяются друг через друга, может возникнуть впечатление, что такое определение является циклическим. Однако, надо заметить, что в определениях используются только предыдущие (по отношению к $<$ и $\triangleleft$ события). Корректность такого определения гарантируется с помощью Нётеровой индукции, которая в свою очередь обеспечивается фундированностью $\triangleleft$ и следующим свойством:

Утверждение 4.1.

Пусть e, e' – события, а $C \in \mathfrak{C}_{fin}$.

1. Если $e < e'$, то $e \triangleleft e'$.
2. Если $e \in C \triangleleft [e']$, то $e \triangleleft e'$.

Также хочется заметить, что в стандартном случае, когда третий компонент контекста срезания определяется как $\{\mathfrak{C}_{loc}\}_{e \in E}$, определение событий-отсечек принимает следующий вид: событие e принадлежит cut_κ , если $e \in fsble_\kappa$ и существует такое событие $e' \in fsble_\kappa$, что $[e'] \approx [e]$ и $e' \triangleleft e$.

После определения возможных и событий-отсечек, естественным образом возникает определение *канонического префикса развертки*: канонические префикс B^κ развертки $U(N)$ – это префикс, содержащий только возможные события $fsble_\kappa$ и смежные к ним позиции. Таким образом, канонический префикс задан однозначно для контекста срезания κ . Важным свойством канонического префикса является его полнота:

Теорема 4.1.

B^κ полон по отношению к множеству событий cut_κ .

Также для канонического префикса выполняется условие конечности:

Теорема 4.2.

B^κ является конечным только в том случае, если в $fsble_\kappa$ не существует бесконечной $<$ -цепочки.

4.2 Алгоритм генерации канонического префикса

В этом разделе приводится алгоритм построения конечного префикса разверток для NP-сетей. Этот алгоритм является модификацией ERV-алгоритма [11]. Алгоритм приведен на рис. 19. Этот алгоритм будем называть базовым.

Базовый алгоритм почти не отличается от алгоритма, приведенного в [17]. Алгоритм из [17] является модификацией ERV-алгоритма, но если оригинальный ERV-алгоритм использовал конкретное определение событий-отсечек, то алгоритм из [17] использует обобщенное алгоритмнезависимое понятия события-отсечки, основанное на теории о канонических префиксов, которая была приведена в прошлой секции. Наш алгоритм отличается функцией возможных расширений $PotExt$. В нашем случае процедура $PotExt$ возвращает все возможные расширения ветвящегося процесса в соответствии с правилами из определ. 3.2.

Корректность алгоритма. В [17] было доказано, что базовый алгоритм строит полные канонические префиксы для классических сетей Петри. Все наши определения, включая определения возможного расширения, удовлетворяют всем требованиям, на которых основано доказательство в [17]. Таким образом, оригинальное доказательство может быть использовано дословно для доказательства корректности базового алгоритма.

Примеры верификации. В качестве примера рассмотрим канонический префикс для сети NP_2 (рис. 13 и 14). В качестве контекста срезания возьмем контекст срезания Мак-Миллана ($C' \approx C'' \iff Mark(C') = Mark(C'')$ и $C' \triangleleft C'' \iff |C'| < |C''|$), т.е. наш префикс будет содержать все возможные достижимые разметки. Результат применения алгоритма показан на рис. 20.

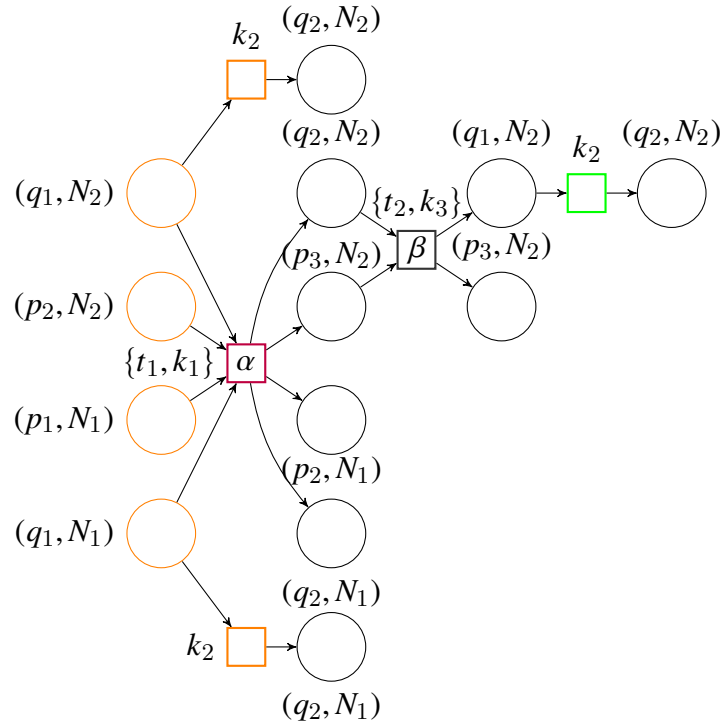


Рис. 20: BP_c : Ветвящийся процесс NP_2

Событие e_c (на рис. 20 отмечено зеленым цветом) помечается как событие-отсечка. Это происходит потому, что конфигурация $[e_c]$ соответствует маркировке, при которой две сетевые фишки находятся в позициях p_2 и p_3 , причем обе фишки имеют одну и ту же маркировку $\{q_2\}$, и при этом в сети существует локальная конфигурация (конфигурация $[e']$, где e' – помеченное фиолетовым цветом на рисунке событие), которая меньше по размеру и соответствует такой же маркировке.

Полученный канонический префикс BP_c позволяет решать проблему выполнимости: может ли когда-нибудь в будущем в сети выполниться данный переход t . Для этого до-

статочно проверить, есть ли в BP_c переход помеченный t , поскольку в конечном префиксе развертки содержаться только те переходы которые могут сработать.

Также с помощью конечных префиксов, используя приведенный выше контекст среза-ния МакМиллана, можно решить проблему дедлока. В сети есть дедлок в том и только в том случае, если в префиксе есть конфигурация не имеющая событий-отсечек, соответствующая маркировка которой является дедлоком. Пример такой конфигурации показан на рис. 20 отмеченной оранжевым цветом. Этой конфигурации соответствует маркировка, в которой две сетевые фишки находятся в позициях p_1 и p_2 , при этом обе сетевые фишки имеют одну и ту же маркировку q_2 . Как можно заметить, такая ситуация является дедлоком.

Приведенный пример показывает, что определенная нами конструкция позволяет проверять важные с точки зрения верификации поведенческие свойства системы.

```

1: procedure UNFOLD( $NP$ )
2:    $Pref \leftarrow$  Пустая сеть ▷ Канонический префикс
3:   Добавить все позиции из начальной маркировке  $NP$  в  $Pref$ , получив тем самым
   начальный ветвящийся процесс  $NP$ 
4:    $pe \leftarrow \text{PotExt}(Pref)$ 
5:    $cut\_off \leftarrow \emptyset$ 
6:   while  $pe \neq \emptyset$  do ▷ Пока существуют возможные расширения ветвящегося
   процесса
7:      $e \leftarrow \min_{\triangleleft} pe$  ▷ Возможен недетерминизм
8:     if  $[e] \cap cut\_off = \emptyset$  then
9:        $Pref \leftarrow Pref \oplus \{e\}$ 
10:       $pe \leftarrow \text{PotExt}(Pref)$ 
11:      if  $e \in cut_{\kappa}$  then ▷ Добавить  $e$  во множество событий-отсечек, если оно
        таковым является
12:         $cut\_off \leftarrow cut\_off \cup \{e\}$ 
13:      end if
14:    else
15:       $pe \leftarrow pe \setminus \{e\}$ 
16:    end if
17:  end while
18:   $Pref \leftarrow Pref \oplus cut\_off$ 
19:  return  $Pref$ 
20: end procedure

```

Рис. 19: Базовый алгоритм

Заключение

В этой работе было представлено конструктивное определение ветвящихся процессов консервативных вложенных сетей Петри. Развертка NP-сети определяется как максимальный ветвящийся процесс. Было показано что такое определение развертки является корректным и полезным поскольку, во-первых, развертка сохраняет поведение сети, во-вторых, к таким определениям развертки и ветвящихся процессов можно легко, почти без изменений, применить теорию канонических префиксов и получить алгоритм построения конечного префикса развертки, с помощью которого можно проводить анализ вложенной сети.

В качестве направления дальнейших исследований рассматриваются два пути:

1. Расширение определения развертки на другие подклассы вложенных сетей Петри.
2. Поиск аксиоматического определения развертки, которое позволяет строить развертку композиционно.

Список использованных источников

- [1] Marek A. Bednarczyk, Luca Bernardinello, Wiesław Pawłowski, and Lucia Pomello. Modelling mobility with petri hypernets. In *Recent Trends in Algebraic Development Techniques*, volume 3423 of *Lecture Notes in Computer Science*, pages 28–44. Springer, 2005.
- [2] Gérard Boudol, Ilaria Castellani, Matthew Hennessy, Mogens Nielsen, and Glynn Winskel. Twenty years on: Reflections on the CEDISYS project. combining true concurrency with process algebra. In Pierpaolo Degano, Rocco Nicola, and José Meseguer, editors, *Concurrency, Graphs and Models*, volume 5065 of *Lecture Notes in Computer Science*, pages 757–777. Springer, 2008.
- [3] Nina Buchina and Leonid Dworzański. The tool for modeling of wireless sensor networks with nested Petri nets. In *Proceedings of 7th the Spring/Summer Young Researchers' Colloquium on Software Engineering, SYRCoSE '13*, pages 15–18. Institute for System Programming of the Russian Academy of Sciences, 2013.
- [4] Thomas Chatain and Claude Jard. Complete finite prefixes of symbolic unfoldings of safe time Petri nets. In Susanna Donatelli and P.S. Thiagarajan, editors, *Petri Nets and Other Models of Concurrency - ICATPN 2006*, volume 4024 of *Lecture Notes in Computer Science*, pages 125–145. Springer, 2006.
- [5] Jean-Michel Couvreur, Sébastien Grivet, and Denis Poitrenaud. Designing a LTL model-checker based on unfolding graphs. In Mogens Nielsen and Dan Simpson, editors, *Application and Theory of Petri Nets 2000*, volume 1825 of *Lecture Notes in Computer Science*, pages 123–145. Springer, 2000.
- [6] Michel Diaz, editor. *Petri nets: fundamental models, verification and applications*. Wiley-ISTE, 2009. 1-656 pp.
- [7] Leonid W. Dworzański and Irina A. Lomazova. On compositionality of boundedness and liveness for nested Petri nets. *Fundamenta Informaticae*, 120(3):275–293, 2012.
- [8] Joost Engelfriet. Branching processes of Petri nets. *Acta Informatica*, 28(6):575–591, 1991.

- [9] Javier Esparza. A false history of true concurrency: From Petri to tools. In *Proceedings of the 17th International SPIN Conference on Model Checking Software*, SPIN'10, pages 180–186. Springer-Verlag, 2010.
- [10] Javier Esparza and Keijo Heljanko. *Unfoldings: a partial-order approach to model checking*. Springer, 2008.
- [11] Javier Esparza, Stefan Römer, and Walter Vogler. An improvement of McMillan’s unfolding algorithm. In Tiziana Margaria and Bernhard Steffen, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, volume 1055 of *Lecture Notes in Computer Science*, pages 87–106. Springer, 1996.
- [12] Javier Esparza and Stefan Römer. An unfolding algorithm for synchronous products of transition systems. In *CONCUR’99 Concurrency Theory*, volume 1664 of *Lecture Notes in Computer Science*, pages 2–20. Springer, 1999.
- [13] Kees M. van Hee, Irina A. Lomazova, Olivia Oanea, Alexander Serebrenik, Natalia Sidorova, and Marc Voorhoeve. Nested nets for adaptive systems. In *Petri Nets and Other Models of Concurrency-ICATPN 2006*, pages 241–260. Springer, 2006.
- [14] Keijo Heljanko. Deadlock and reachability checking with finite complete prefixes. Technical report, Helsinki University of Technology, Laboratory for Theoretical Computer Science, 1999.
- [15] Kathrin Hoffmann, Hartmut Ehrig, and Till Mossakowski. High-level nets with nets and rules as tokens. In Gianfranco Ciardo and Philippe Darondeau, editors, *Applications and Theory of Petri Nets 2005*, volume 3536 of *Lecture Notes in Computer Science*, pages 268–288. Springer, 2005.
- [16] Kurt Jensen and Lars M. Kristensen. *Coloured Petri nets: modelling and validation of concurrent systems*. Springer, 2009. 384 pp.
- [17] Victor Khomenko. *Model Checking Based on Prefixes of Petri Net Unfoldings*. Ph.D. Thesis, School of Computing Science, Newcastle University, 2003.
- [18] Victor Khomenko and Maciej Koutny. Branching processes of high-level Petri nets. In Hubert Garavel and John Hatcliff, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, volume 2619 of *Lecture Notes in Computer Science*, pages 458–472. Springer, 2003.

- [19] Victor Khomenko, Maciej Koutny, and Walter Vogler. Canonical prefixes of Petri net unfoldings. *Acta Informatica*, 40(2):95–118, 2003. ISSN 0001-5903.
- [20] Michael Köhler and Berndt Farwer. Object nets for mobility. In Jetty Kleijn and Alex Yakovlev, editors, *Petri Nets and Other Models of Concurrency – ICATPN 2007*, volume 4546 of *Lecture Notes in Computer Science*, pages 244–262. Springer, 2007.
- [21] Rom Langerak and Ed Brinksma. A complete finite prefix for process algebra. In Nicolas Halbwachs and Doron Peled, editors, *Computer Aided Verification*, volume 1633 of *Lecture Notes in Computer Science*, pages 184–195. Springer, 1999.
- [22] Irina A. Lomazova. Nested Petri nets—a formalism for specification and verification of multi-agent distributed systems. *Fundamenta Informaticae*, 43(1):195–214, 2000.
- [23] Irina A. Lomazova. Modeling dynamic objects in distributed systems with nested Petri nets. *Fundamenta Informaticae*, 51(1-2):121–133, 2002.
- [24] Irina A. Lomazova. Nested Petri nets for adaptive process modeling. In *Pillars of computer science*, pages 460–474. Springer, 2008.
- [25] Marco Mascheroni. Generalized hypernets and their semantics. In *Proceedings of the Fifth International Workshop on Modelling of Objects, Components and Agents, MOCA*, volume 9, pages 87–106, 2009.
- [26] Marco Mascheroni. *Hypernet: a Class of Hierarchical Petri Nets*. Ph.D. Thesis, Facoltà di Scienze Naturali Fisiche e Naturali, Dipartimento di Informatica Sistemistica e Comunicazione, Università Degli Studi Di Milano Bicocca, 2010.
- [27] Kenneth L. McMillan. Using unfoldings to avoid the state explosion problem in the verification of asynchronous circuits. In Gregor Bochmann and David Karl Probst, editors, *Computer Aided Verification*, volume 663 of *Lecture Notes in Computer Science*, pages 164–177. Springer, 1993.
- [28] Kenneth L. McMillan. A technique of state space search based on unfolding. *Form. Methods Syst. Des.*, 6(1):45–65, 1995.
- [29] Mogens Nielsen, Gordon Plotkin, and Glynn Winskel. Petri nets, event structures and domains, part I. *Theoretical Computer Science*, 13(1):85–108, 1981.

- [30] Carl A. Petri. *Kommunikation mit Automaten*. Ph.D. Thesis, Institut für instrumentelle Mathematik, Bonn, 1962.
- [31] Wolfgang Reisig. *Understanding Petri Nets - Modeling Techniques, Analysis Methods, Case Studies*. Springer, 2013. 1-230 pp.
- [32] Fernando Rosa-Velardo. Petri nets with name creation for transient secure association. *Acta Informatica*, 50(7-8):403–436, 2013. ISSN 0001-5903.
- [33] Rüdiger Valk. Petri nets as token objects: An introduction to Elementary Object Nets. In Jörg Desel and Manuel Silva, editors, *Application and Theory of Petri Nets 1998*, volume 1420 of *Lecture Notes in Computer Science*, pages 1–24. Springer, 1998.
- [34] Rüdiger Valk. Object Petri nets. In Jörg Desel, Wolfgang Reisig, and Grzegorz Rozenberg, editors, *Lectures on Concurrency and Petri Nets*, volume 3098 of *Lecture Notes in Computer Science*, pages 819–848. Springer, 2004.
- [35] Ирина Александровна Ломазова. *Вложенные сети Петри: моделирование и анализ распределенных систем с объектной структурой*. Научный Мир, Москва, 2004. 208 pp.